

Projekt: Software Engineering

Finalisierungsphase

Master Informatik

09.06.2026

Cornelius Rottmair

Matrikelnummer: IU14120646

Tutor: Dirk Simon

Abstrakt

Einleitung und Projektziele

Ziel des Projekts war die Entwicklung einer nativen Windows-Desktop-Anwendung, die lokale Bild- und Videodateien semantisch durchsuchbar macht. Im Gegensatz zu klassischen Ansätzen, die auf Dateinamen oder manuellen Metadaten basieren, sollte die Anwendung den Inhalt von Medien mithilfe vortrainierter multimodaler KI-Modelle verstehen und vergleichen. Der Kundenwert liegt in der Kombination aus Zeitersparnis und lokalem Datenschutz: Nutzer sollen große Mediensammlungen per Texteingabe oder Referenzbild durchsuchen können, ohne ihre Daten an Cloud-Dienste zu übertragen. Als technische Grundlage wurde BLIP (Bootstrapping Language-Image Pre-training) gewählt, das sowohl Bildvektorisierung als auch automatische Bildbeschreibungen ermöglicht.

Entwicklungsverlauf und unerwartete Probleme

Die Entwicklung erfolgte unter WSL (Windows Subsystem for Linux), da die Entwicklungsumgebung bereits installiert war. Die eigentliche Implementierung verlief unter Linux problemlos und zügig: Python, PySide6 und die BLIP-Modelle ließen sich erstaunlich einfach integrieren, genau wie die Anbindung der HuggingFace-Modelle. Die eigentlichen Schwierigkeiten traten erst beim Build-Prozess auf. Da die Anwendung als native Windows-EXE ausgeliefert werden sollte, wurde PyInstaller über einen GitHub-Actions-Workflow auf einem Windows-Runner kompiliert. Diese Entscheidung hatte einen erheblichen Mehraufwand zur Folge: Fehler, die unter Linux nicht auftraten, zeigten sich erst nach dem Kompilierungsvorgang auf GitHub – beispielsweise Sonderzeichen in Pfaden, die unter Windows nicht zulässig sind, fehlende Schreibrechte für den Modellordner sowie inkompatible CUDA-Versionen zwischen PyTorch und den installierten Treibern. Jeder dieser Fehler erforderte einen neuen Kompilierungsdurchlauf, was aufgrund der Buildzeit von mehreren Minuten pro Durchlauf auf GitHub zeitaufwendig war. Im Rückblick wäre es deutlich effizienter gewesen, die Entwicklungsumgebung von Anfang an nativ unter Windows aufzusetzen, um Plattformunterschiede frühzeitig zu erkennen.

Technische Herausforderungen: Textsuche und Modality Gap

Eine der zentralen Herausforderungen stellte der Vergleich von Texteingaben mit Bildvektoren dar. Der ursprüngliche Ansatz, den Nutzer-Prompt direkt als Vektor mit den Bildvektoren zu vergleichen, lieferte in der Praxis teilweise unsinnige Ergebnisse: Selbst wenn der KI-Prompt eines Bildes den exakt gesuchten Gegenstand oder das gesuchte Tier erkannt hatte, war der Vektor eines Bildes ohne den gesuchten Inhalt manchmal dennoch näher am Suchvektor, also der Modality Gap.

Die implementierte Lösung umgeht dieses Problem, indem der Nutzer-Prompt nicht mit dem Bildvektor, sondern per Schlagwortabgleich mit der generierten Caption verglichen wird (siehe `calculate_strict_keyword_score` in `engine/ai_worker.py`). Für jedes Wort des bereinigten Prompts wird geprüft, ob es in den Wörtern der Caption vorkommt; der resultierende Anteil bestimmt den Score. Diese Lösung funktionierte in der Praxis recht zuverlässig, hat jedoch Grenzen bei abstrakten Anfragen oder Synonymen, die nicht im generierten Caption-Vokabular auftreten.

Positive Erkenntnisse

Trotz der genannten Probleme zeigte das Projekt, dass die Integration vortrainierter KI-Modelle in Desktop-Anwendungen mit modernen Python-Bibliotheken technisch einfach umzusetzen ist. Die gewählte Schichtenarchitektur ermöglichte es, die Suchlogik mehrfach grundlegend zu überarbeiten, ohne die Benutzeroberfläche anpassen zu müssen. Ebenso war die Auslagerung rechenintensiver Aufgaben in QThread-Instanzen eine gute Lösung, da die GUI auch bei längerer Modellverarbeitung jederzeit reaktionsfähig bleibt.

Was ich beim nächsten Projekt anders machen würde

Bei Projekten, deren Zielplattform Windows ist, sollte die Entwicklungsumgebung ebenfalls von Anfang an unter Windows eingerichtet werden, auch wenn Linux im Umgang oft einfacher ist. Die Kosten später auftretender Plattformfehler, insbesondere wenn jeder Fix einen vollständigen CI-Build erfordert, überwiegen den anfänglichen Mehraufwand vermutlich bei Weitem. Technisch würde ich früher eine lokale Windows-Testumgebung einrichten und den Build-Prozess bereits während der Entwicklungsphase regelmäßig ausführen, anstatt ihn erst am Ende zu validieren.

Fazit

Die Anwendung erfüllt die gesetzten Kernziele: Lokale semantische Suche in Bild- und Videodateien ist möglich, alle Verarbeitungsschritte verbleiben auf dem Gerät des Nutzers, und die Benutzeroberfläche bleibt auch bei laufender KI-Verarbeitung stabil. Das Projekt hat verdeutlicht, dass der Mehrwert moderner KI-Modelle nicht nur in ihrer Leistungsfähigkeit liegt, sondern auch darin, wie sie in eine nutzbare und erklärbare Anwendung eingebettet werden. Genau diese Integration, zwischen Modellausgabe, Benutzeroberfläche und verständlichem Ergebnis, war ja das eigentliche Ziel dieses Projekts.

Inhaltsverzeichnis

1	Konzeptionsphase	1
1.1	Projektdokumentation	1
1.1.1	Projektübersicht	1
1.1.2	Risikomanagement	1
1.1.3	Zeitplanung	4
1.2	Anforderungsdokument	5
1.2.1	Stakeholder	5
1.2.2	Funktionale Anforderungen)	5
1.2.3	Nicht-funktionale Anforderungen (NFRs)	7
1.2.4	Glossar	7
1.3	Spezifikationsdokument	8
1.4	Theoretische Grundlagen	8
1.4.1	Datenmodell	8
1.4.2	Geschäftsprozesse	9
1.4.3	Geschäftsregeln	10
1.4.4	Systemschnittstellen	11
1.4.5	Benutzerschnittstellen (GUI)	11
2	Erarbeitungs- und Reflexionsphase	13
2.1	Architekturdokument	13
2.1.1	Technologieübersicht	13
2.1.2	Architekturübersicht	13
2.1.3	Strukturübersicht	15
2.1.4	Verhaltensübersicht	17
2.2	GitHub	17

2.3	Benutzeranleitung	18
2.3.1	Semantische Suche für lokale Medien	18
2.3.2	Eingabe	18
2.3.3	Suche	19
3	Finalisierungsphase	19
3.1	Teststrategie	19
3.2	Unit-Tests	21
3.3	Integrationstests	24
3.4	Systemtests	26

Abbildungsverzeichnis

1	Risikomatrix	3
2	Use-Case-Übersicht des Systems	6
3	UML-Diagramm	9
4	Aktivitätsdiagramm	10
5	GUI	11
6	UML-Klassendiagramm	15
7	UML-Sequenzdiagramm	17

1 Konzeptionsphase

1.1 Projektdokumentation

1.1.1 Projektübersicht

Das Ziel des Projekts ist die Entwicklung einer Desktop Anwendung für Windows, die es Benutzern ermöglicht soll, lokal Bild- und Videodateien semantisch zu durchsuchen. Im Gegensatz zu herkömmlichen Methoden, die auf manuell vergebenen Metadaten oder Dateinamen basieren, soll die Anwendung künstliche Intelligenz nutzen, um die Inhalt der Medien zu “verstehen” und auf dieser Basis entscheidungen treffen.

Die Anwendung soll es Nutzern, über Text Prompts oder Referenzbilder gezielt nach Medieninhalten zu suchen. Dabei sollen multimedialen Transformer Modelle eingesetzt werden, die Bilder und Videos in Repräsentationen(Vektoren) umwandeln, sodass Vergleiche zwischen Medien oder zwischen Nutzer Prompt und Modell möglich sind.

Der zentrale Customer Value liegt in der Zeitersparnis und dem Datenschutz: Nutzer sollen durch einfache Suchanfragen (z. B. "weiß graue Katze") oder durch Referenzbilder gesuchte Inhalte in großen Datenmengen finden, ohne ihre privaten Daten an Cloud Dienste senden zu müssen.

1.1.2 Risikomanagement

Um die Ziele eines Projekts einzuhalten, sollten mögliche Störungen vorher erfasst und analysiert werden, um mögliche Gegenmaßnahmen treffen zu können. Im Risikomanagement werden diese Risiken nach ihrem Schadensausmaß (SA) und ihrer Eintrittswahrscheinlichkeit (EW) bewertet, um sie miteinander zu vergleichen Atlassian (2026).

- **RK_1: Hoher Ressourcenverbrauch (RAM / GPU).** Da KI-Modelle rechenintensiv sind, kann die Anwendung auf ressourcenschwachen PCs abstürzen oder einfrieren.
 - **EW:** mittel **SA:** mittel
 - *Gegenmaßnahme:* Bereitstellung unterschiedlich großer Modelle sowie Auslagerung rechenintensiver Prozesse in Hintergrund-Threads, um eine Blockierung der grafischen Benutzeroberfläche zu vermeiden.

- **RK_2: Lange Verarbeitungszeit bei großen Mediendatensätzen.** Beim Analysieren umfangreicher Bild- oder Videosammlungen können längere Wartezeiten auftreten, was die Nutzerakzeptanz beeinträchtigen kann.
 - **EW:** mittel **SA:** gering
 - *Gegenmaßnahme:* Einsatz von Fortschrittsanzeigen und Visualisierung des aktuellen Verarbeitungsstatus.
- **RK_3: Nachvollziehbarkeit von Modellentscheidungen.** Warum das Modell zwei Bilder als ähnlich oder unähnlich einstuft, ist oft überhaupt nicht nachvollziehbar und kann teilweise komplett unnachvollziehbar sein Qamar and Bawany (2023).
 - **EW:** sehr hoch **SA:** sehr hoch
 - *Gegenmaßnahme:* So erklärbar wie möglich machen, z. B. durch automatisch generierte Bildbeschreibungen, die die Entscheidungsgrundlage verdeutlichen.
- **RK_4: Fehlinterpretation von Suchtreffern bei Videos.** Suchtreffer basieren häufig auf einzelnen Frames und repräsentieren nicht zwangsläufig den gesamten Videoinhalt.
 - **EW:** mittel **SA:** gering
 - *Gegenmaßnahme:* Darstellung des relevanten Zeitpunkts im Video sowie eine Vorschau des entsprechenden Frames.
- **RK_5: Abhängigkeit von vortrainierten Modellen.** Die Qualität der Suchergebnisse hängt stark von den eingesetzten vortrainierten Modellen ab. Transfer-Learning-Ansätze können bei Domänenunterschieden zwischen den Trainingsdaten und den tatsächlichen Nutzerdaten zu Leistungseinbußen führen (Iman et al., 2023, S. 2).
 - **EW:** hoch **SA:** hoch
 - *Gegenmaßnahme:* Verwendung einer modularen Architektur, die den Austausch oder die Erweiterung der Modelle ermöglicht.
- **RK_6: Eingeschränkte Generalisierbarkeit von Suchanfragen.** Bestimmte Suchanfragen können von den Modellen nicht zuverlässig erfasst werden.
 - **EW:** sehr hoch **SA:** hoch

- *Gegenmaßnahme*: Kombination von Text- und Bildsuche sowie transparente Kommunikation der Systemgrenzen gegenüber den Nutzern.
- **RK_7: Vergleichbarkeit von User-Prompt und Modell-Ausgaben.** Während zwei Medienobjekte direkt im Vektorraum des Modells sinnvoll vergleichbar sind, liefert der Vergleich eines Nutzer Prompts mit dem Bild Vektor des Modells oft sehr niedrige Scores, da der Prompt nicht alle Informationen enthält, die das Modell im Vektor abbildet (Liang et al., 2022, S. 1).
 - **EW**: hoch **SA**: mittel
 - *Gegenmaßnahme*: Verwendung einer alternativen Vergleichsmethode auf Basis von Wortabgleich, um Nutzer-Prompts mit den automatisch generierten Beschreibungen der Medien sinnvoll zu vergleichen.
- **RK_8: Hoher Speicherbedarf durch mehrere Modelle.** Die Bereitstellung oder Nutzung mehrerer Modelle gleichzeitig kann sehr viel Speicherplatz beanspruchen, insbesondere wenn der Nutzer ohnehin nur ein bestimmtes Modell verwendet.
 - **EW**: mittel **SA**: gering
 - *Gegenmaßnahme*: Möglichkeit für den Nutzer, auszuwählen, welche Modelle heruntergeladen oder genutzt werden sollen, um Speicherressourcen zu sparen.

Risikomatrix						
Eintrittswahrscheinlichkeit (EW)	sehr hoch 5				RK_6	RK_3
	hoch 4			RK_7	RK_5	
	mittel 3		RK_2, RK_4, RK_8	RK_1		
	niedrig 2					
	sehr niedrig 1					
		1 sehr gering	2 gering	3 mittel	4 hoch	5 sehr hoch
Schadensausmaß (SA)						

Abbildung 1: Risikomatrix

1.1.3 Zeitplanung

- **Meilenstein 1: Projektstart**

- Zielsetzung und Anforderungen der Anwendung im Konzeptionspapier festlegen
- Funktionsumfang, Datenschutz und Ressourcenanforderungen im Konzeptionspapier definieren

- **Meilenstein 2: Softwarearchitektur und Design**

- Entwurf der modularen Softwarearchitektur
- GUI Design und Nutzerinteraktionen skizzieren
- Auswahl der KI-Modelle treffen

- **Meilenstein 3: Grundgerüst der Anwendung implementieren**

- Entwicklung der grafischen Oberfläche
- Hintergrundverarbeitung integrieren

- **Meilenstein 4: Integration der KI-Modelle**

- Modelle einbinden
- Funktionen für Bild zu Bild (vektor zu Vektor)- und Text zu Bild (Text zu Vektor) Vergleich implementieren

- **Meilenstein 5: Optimierung**

- Modellauswahl für Nutzer einbauen
- Ressourcen- und Speicherverwaltung implementieren
- Automatische Beschreibungen zur Explainability hinzufügen

- **Meilenstein 6: Testphase**

- Funktionstests und Performancetests durchführen
- Usabilitytests der GUI
- Fehlerbehebung und Optimierung

- **Meilenstein 7: Deployment**

- Benutzer- und technische Dokumentation fertigstellen
- Anwendung bereitstellen

1.2 Anforderungsdokument

1.2.1 Stakeholder

- **Privatnutzer mit großen Mediensammlungen:** Personen, die Terabytes an Fotos/Videos lokal speichern und ohne Cloud durchsuchen wollen.
- **Content Creator/Fotografen:** Nutzer, die visuelle Referenzen (z. B. "ähnliches Bild wie dieses") auf ihrer Festplatte finden müssen.
- **Datenschutz Fokussierte Anwender:** Nutzer, die explizit KI gestützte Suche nutzen wollen, aber die Verarbeitung sensibler Daten auf dem eigenen Gerät voraussetzen.

1.2.2 Funktionale Anforderungen)

ID	User Story (Funktionale Anforderung)	Priorität
US-01	Medien-Indizierung: Als Nutzer möchte ich lokale Verzeichnisse auswählen, die rekursiv nach unterstützten Formaten durchsucht werden.	Must-Have
US-02	Visuelle Suche: Als Nutzer möchte ich ein Referenzbild per Drag-and-Drop nutzen, um visuell ähnliche Medien zu finden.	Must-Have
US-03	Textsuche: Als Nutzer möchte ich über Text-Prompts suchen, wobei das System primär Schlüsselwörter mit den generierten Bildbeschreibungen abgleicht.	Must-Have
US-04	Video-Analyse: Als Nutzer möchte ich, dass Videos über extrahierte Frames analysiert werden und das Suchergebnis den exakten Zeitstempel ausgibt.	Should-Have
US-05	Explainability: Als Nutzer möchte ich zu jedem Bild eine vom Modell generierte Beschreibung sehen, um die Relevanz des Treffers nachzuvollziehen.	Should-Have
US-06	UI-Feedback: Als Nutzer möchte ich durch UI-Sperren (z. B. ausgegraute Eingabefelder) eindeutig erkennen, welcher Suchmodus aktuell aktiv ist.	Could-Have
US-07	Ressourcenmanagement: Als Nutzer möchte ich das verwendete KI-Modell manuell auswählen können, um Speicherplatz und Rechenleistung zu skalieren.	Could-Have

Tabelle 1: Priorisierung der funktionalen Anforderungen nach dem MoSCoW-Prinzip

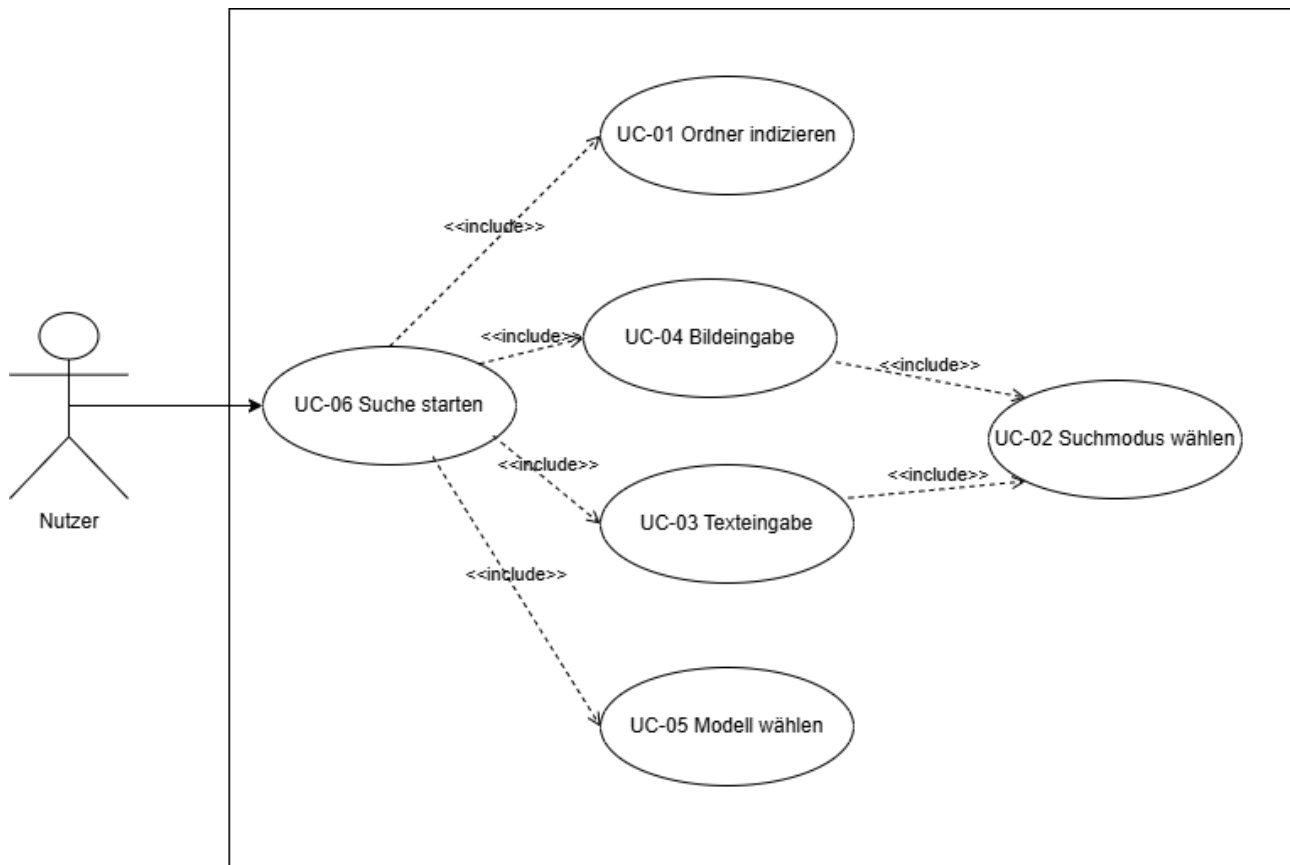


Abbildung 2: Use-Case-Übersicht des Systems

UC-ID	Name	Akteur	Vorbedingung	Ergebnis
UC-01	Ordner indizieren	Nutzer	Anwendung gestartet	Medienpfade in Warteschlange geladen
UC-02	Suchmodus wählen	Nutzer	Anwendung gestartet	Suchmodus (Text/Bild) ist aktiv
UC-03	Texteingabe	Nutzer	Modus „Textsuche“ gewählt	Such-Prompt wurde eingegeben
UC-04	Bildeingabe	Nutzer	Modus „Bildsuche“ gewählt	Bild für die Suche wurde ausgewählt
UC-05	Modell wählen	Nutzer	Anwendung gestartet	Spezifiziertes Modell ist für Suche geladen
UC-06	Suche starten	Nutzer	UC-01, UC-02, UC-05 und UC-03/04	Trefferliste inklusive Score und Caption

Tabelle 2: Übersicht der Use-Cases und deren Abhängigkeiten

1.2.3 Nicht-funktionale Anforderungen (NFRs)

ID	Kategorie	Spezifikation
NFA-01	Datenschutz	Es darf keine Datenübertragung an externe Server stattfinden.
NFA-02	Performance	Inferenzprozesse werden vom Main-UI Thread entkoppelt, um ein mögliches Einfrieren der GUI zu verhindern.
NFA-03	Skalierbarkeit	Das System muss in der Lage sein, Archive im Umfang von zehntausenden Elementen performant abzufragen.
NFA-04	Wartbarkeit	Die Anwendung sollte modular konzipiert sein, sodass eingesetzte KI-Modelle über definierte Schnittstellen ohne Refactoring ausgetauscht werden können.
NFA-05	Usability	Die Anwendung ist als eigenständig ausführbare Datei (.exe) unter Windows bereitzustellen.

Tabelle 3: Spezifikation der nicht-funktionalen Anforderungen

1.2.4 Glossar

Multimediale Transformer: Komplexe KI-Architekturen, die unterschiedliche Modalitäten (Bild und Text) verarbeiten und in einen gemeinsamen semantischen Vektorraum abbilden.

Vektor: Eine hochdimensionale numerische Repräsentation, die die Merkmale eines Mediums kodiert. Eine hohe Vektor-Ähnlichkeit indiziert eine hohe semantische Relevanz.

Prompt: Eine textuelle Eingabe, die intern in einen Vektor transformiert wird, um mathematische Abgleiche mit den Datenbank-Vektoren der indizierten Medien zu ermöglichen.

Semantische Suche: Ein Paradigma der Informationsrecherche, welches Inhalte anhand ihrer Bedeutung und ihres Kontextes analysiert, statt lediglich auf Metadaten oder Dateinamen zurückzugreifen.

1.3 Spezifikationsdokument

1.4 Theoretische Grundlagen

Wie im Anforderungsdokument beschrieben benutzt die Anwendung *Vision-Language Pre-training* (VLP), bei dem Modelle auf großen Bild-Text-Datensätzen trainiert werden, um einen gemeinsamen Einbettungsraum für visuelle und sprachliche Inhalte zu erlernen.

CLIP (*Contrastive Language-Image Pre-Training*) von Radford et al. Radford et al. (2021) lernt durch kontrastives Training auf 400 Millionen Bild-Text-Paaren, welche Bildmerkmale zu welchen Textbeschreibungen gehören. Bild und Text Encoder werden dabei so optimiert, dass einander semantisch entsprechende Paare im Vektorraum nahe beieinanderliegen. Dadurch wird ein direkter Vektor Vergleich zwischen einem Text Prompt und einem Bild möglich (vgl. US-02, US-03).

BLIP (*Bootstrapping Language-Image Pre-training*) von Li et al. Li et al. (2022) erweitert diesen Ansatz um eine dedizierte *Captioning-Komponente*: Ein generatives Teilmodell erzeugt für jedes Eingabebild eine natürlichsprachliche Beschreibung. Diese Captions werden in der vorliegenden Anwendung eingesetzt, um die Nachvollziehbarkeit von Suchergebnissen zu erhöhen (US-05) und einen Schlagwort Abgleich zu ermöglichen, der die semantische Lücke zwischen reinen Bildvektoren und kurzen Text Prompts überbrückt (R_7).

1.4.1 Datenmodell

- **Medien Warteschlange**: Verwaltet die vom Nutzer ausgewählten Dateipfade für den Suchvorgang.
- **Medium**: Die Basisklasse für die Dateitypen, Bild und Video.
- **Suchkonfiguration**: Speichert die vom Nutzer gesetzten Parameter.
- **Analyse Einheit**: Die zentrale Steuerungskomponente. Sie verarbeitet die Warteschlange unter Berücksichtigung der Such Konfiguration und generiert die Ergebnisse.
- **Suchergebnis**: Repräsentiert einen einzelnen Treffer. Es speichert die berechnete Wahrscheinlichkeit und die generierte Bildbeschreibung.

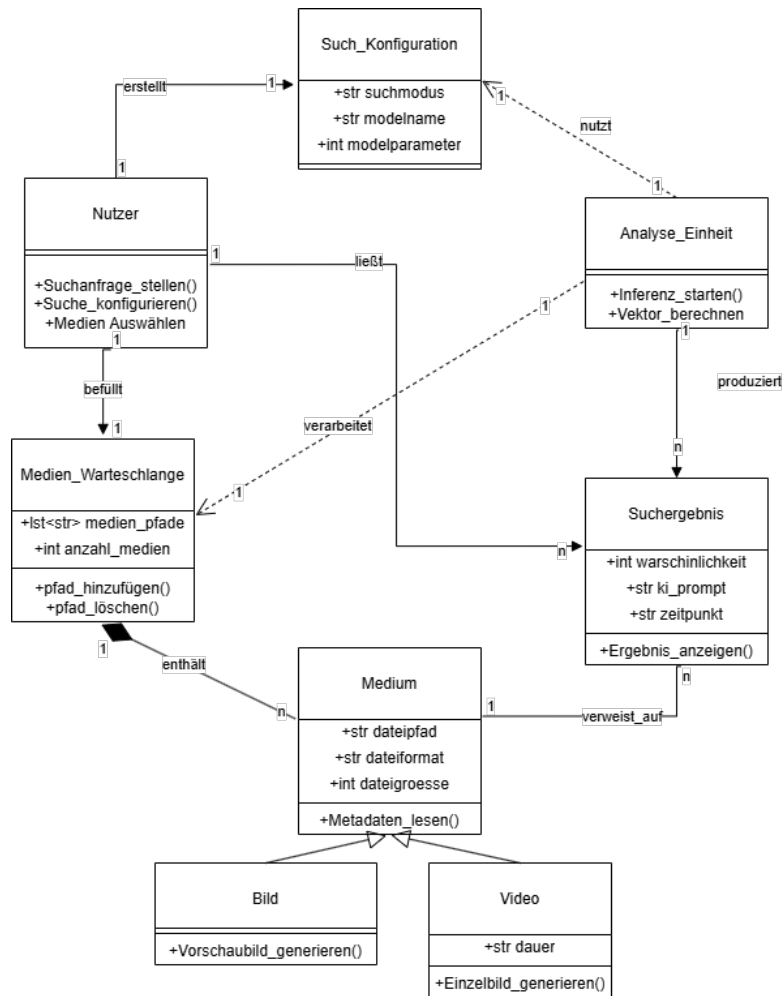


Abbildung 3: UML-Diagramm

1.4.2 Geschäftsprozesse

Um die Suche zu starten, muss der Nutzer zuerst entweder einen Suchprompt mit Schlagwörtern eingeben oder ein Bild hochladen. Falls er sich für das Bild entscheidet, wandelt das Modell zur besseren Erklärung das Bild in einen Textprompt um und zeigt diesen an. Danach muss er die zu durchsuchenden Dateien oder Ordner auswählen sowie die gewünschten Modellparameter festlegen. Stimmen die Eingaben, wird durch die Liste iteriert. Im Fall von Bildern wird ein Vektor erstellt und mit dem eingegebenen Bild abgeglichen. Im Fall von Videos werden die einzelnen Frames des Videos untersucht, und das beste Bild (und dessen Zeit) wird verwendet. Liegt ein Bild über einem Grenzwert, gilt es als Treffer und wird angezeigt. Während des Prozesses sollte die Warteschlange mit Treffern und Nicht Treffern aktualisiert werden. Wenn durch die Liste iteriert wurde, ist der Prozess abgeschlossen.

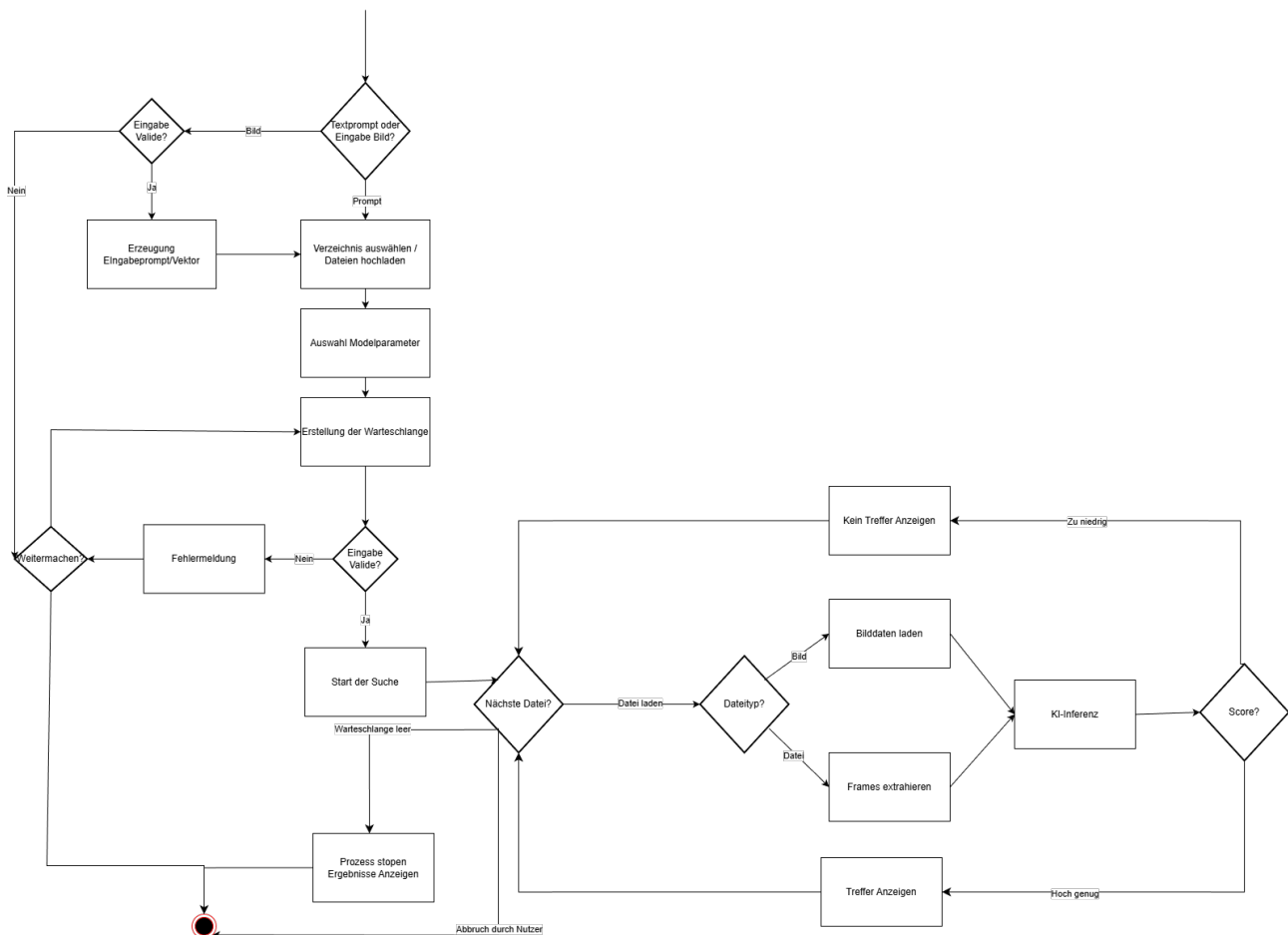


Abbildung 4: Aktivitätsdiagramm

1.4.3 Geschäftsregeln

- **BR-01: Exklusivität der Eingabequelle**

Sobald ein Bild als Suchreferenz verwendet wird, muss das System die manuelle Texteingabe sperren, um Mehrdeutigkeiten bei der Anfrageerstellung zu verhindern. Der Nutzer muss die Bildquelle explizit entfernen, um wieder Text eingeben zu können.

- **BR-02: Intervallbasierte Video Analyse**

Um die Performance zu gewährleisten, werden Videos nicht Bild für Bild analysiert. Das System extrahiert Frames in einem festen zeitlichen Intervall (Standard: 2 Sekunden), um eine repräsentative Abdeckung bei minimaler Rechenlast zu erzielen.

- **BR-03: Suchmodi**

Das System bietet zwei getrennte Suchlogiken an, zwischen denen der Nutzer explizit wählen muss:

1. *Vektor-Raum*: Semantischer Vergleich basierend Modelausgabe vektoren.

2. *Schlagwort Abgleich*: Textbasierter Vergleich zwischen Nutzer Prompt und generierter Bildbeschreibung.

- **BR-04: Relevanzfilterung**

Suchergebnisse werden erst ab einem definierten Ähnlichkeitswert als treffer gewertet. Ergebnisse mit sehr hoher Übereinstimmung müssen visuell hervorgehoben werden.

1.4.4 Systemschnittstellen

- **IF-01: Lokales Dateisystem**

- *Zweck*: Rekursives Einlesen von Bild- und Videodateien aus Nutzerverzeichnissen sowie temporäres Caching von Thumbnails.
- *Format*: Unterstützung gängiger Formate wie JPG, PNG, MP4, MKV.

- **IF-02: GPU**

- *Zweck*: Das System prüft beim Start die Verfügbarkeit einer kompatiblen Grafikkarte und lagert die Tensor Berechnungen automatisch aus, um die Verarbeitungszeit zu minimieren.

1.4.5 Benutzerschnittstellen (GUI)

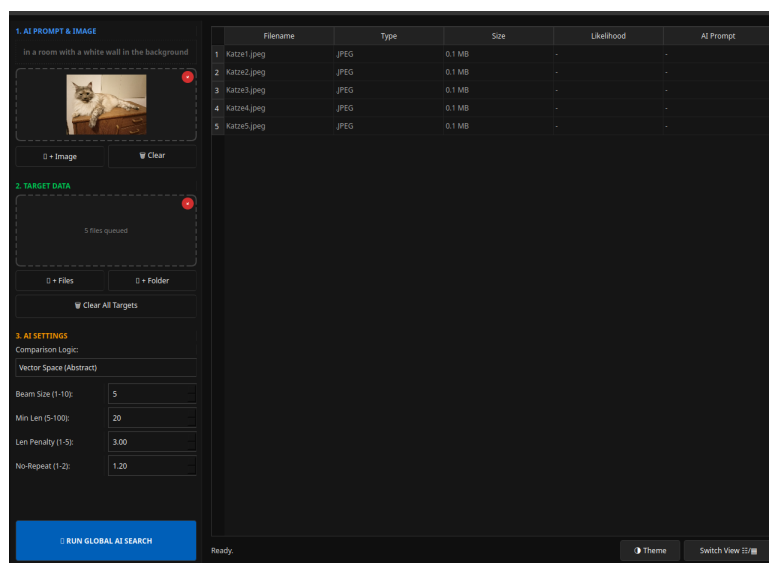


Abbildung 5: GUI

- **GUI-01: Sidebar**

- *Suchinput*: Kombiniertes Feld für Text Prompts und eine "Drop Zone" für Referenzbilder.
- *Zieldaten*: Liste der zu durchsuchenden Dateien/Ordner mit Hinzufügen/Löschen Funktionen.
- *KI-Einstellungen*:
 - * Auswahlmöglichkeit des Modells
 - * Dropdown zur Wahl der Vergleichslogik .
 - * Numerische Eingabefelder zur Modellparameterauswahl

- **GUI-02: Ergebnisdarstellung**

- *View Switch*: Umschaltmöglichkeit zwischen einer detaillierten Tabellenansicht und einer Galerieansicht.
- *Galerie Elemente*: Zeigen ein Vorschaubild, den Ähnlichkeitsscore, bei Videos den Zeitstempel des Treffers sowie die generierte KI Prompts.
- *Feedback*: Während der Verarbeitung zeigen die Kacheln einen aktiven Status, bei Treffern werden sie markiert.

2 Erarbeitungs- und Reflexionsphase

2.1 Architekturdokument

2.1.1 Technologieübersicht

Die nachfolgende Tabelle gibt einen Überblick über alle eingesetzten Technologien sowie die Begründung ihrer Auswahl.

Tabelle 4: Technologieübersicht

Technologie	Rolle im Projekt	Begründung der Auswahl
Python 3	Implementierungssprache	Standard im Bereich maschinelles Lernen PyTorch Contributors (2024).
PySide6	GUI-Framework	Offizielle Python-Bindung für Qt 6 The Qt Company (2024); ermöglicht native Desktop-Anwendungen unter Windows; integriert <code>QThread</code> für nicht-blockierende Hintergrundverarbeitung (vgl. NFA-02).
PyTorch	Modell-Ausführung	Deep-Learning-Framework mit imperativer Programmierschnittstelle Paszke et al. (2019); einfache CPU/GPU-Umschaltung über <code>torch.device</code> (vgl. IF-02).
HuggingFace Transformers	Modellbereitstellung	Stellt vortrainierte BLIP-Gewichte bereit Li et al. (2022); einheitliche API für Laden, Tokenisierung und Inferenz verschiedener Transformer-Modelle.
OpenCV	Videoverarbeitung	Standard für Computer-Vision-Aufgaben in Python OpenCV Team (2026); Gebraucht für das Verarbeiten von MP4/MKV (vgl. US-04, BR-02).
Pillow	Bildverarbeitung	Bibliothek für Bildlade- und Vorverarbeitungsoperationen; komplementär zu OpenCV für Bilder Pillow Contributors (2026).

2.1.2 Architekturübersicht

Die Anwendung ist eine lokale Desktop Anwendung und folgt einer Schichtenarchitektur. Eine Verarbeitung über externe Server oder Cloud-Dienste wird nicht genutzt, da die Daten aus Datenschutzgründen ausschließlich lokal verarbeitet werden sollen (NFA-01).

Das Muster der Schichtenarchitektur trennt eine Anwendung in hierarchisch geordnete Schichten, wobei die Kommunikation zwischen den Schichten über definierte Schnittstellen erfolgt (Marmsoler et al., 2015, S. 47).

Die Architektur besteht aus den drei Schichten:

1. **UI-Schicht:** Verantwortlich für die grafische Oberfläche der Anwendung. Dazu gehören das Hauptfenster (`MainWindow`) sowie spezialisierte UI-Komponenten wie `UniversalCard` und `SmartDropZone`. Die UI-Schicht nimmt keine KI-Berechnungen vor und ist damit vollständig von der Verarbeitungslogik entkoppelt.
2. **Logikschicht:** Enthält die gesamte Verarbeitungslogik der Anwendung. Die Komponenten `AIWorker` und `ModelLoader` führen Modellberechnungen in separaten `QThread`-Instanzen aus, damit die Benutzeroberfläche während der Verarbeitung reaktionsfähig bleibt (NFA-02).
3. **Modellschicht:** Übernimmt den Zugriff auf das lokale Dateisystem sowie die Verwaltung der lokal gespeicherten KI-Modellgewichte. Diese Schicht ist unabhängig von der Darstellungslogik und kann ausgetauscht werden, ohne die UI-Schicht zu verändern (NFA-05, R_5).

Durch diese Aufteilung sind die einzelnen Bereiche voneinander getrennt. Änderungen an der Suchlogik oder an den verwendeten Modellen können dadurch vorgenommen werden, ohne die Benutzeroberfläche ändern zu müssen. Das Hauptfenster koordiniert lediglich die Anzeige, während spezialisierte Worker im Hintergrund die KI-Verarbeitung übernehmen.

2.1.3 Strukturübersicht

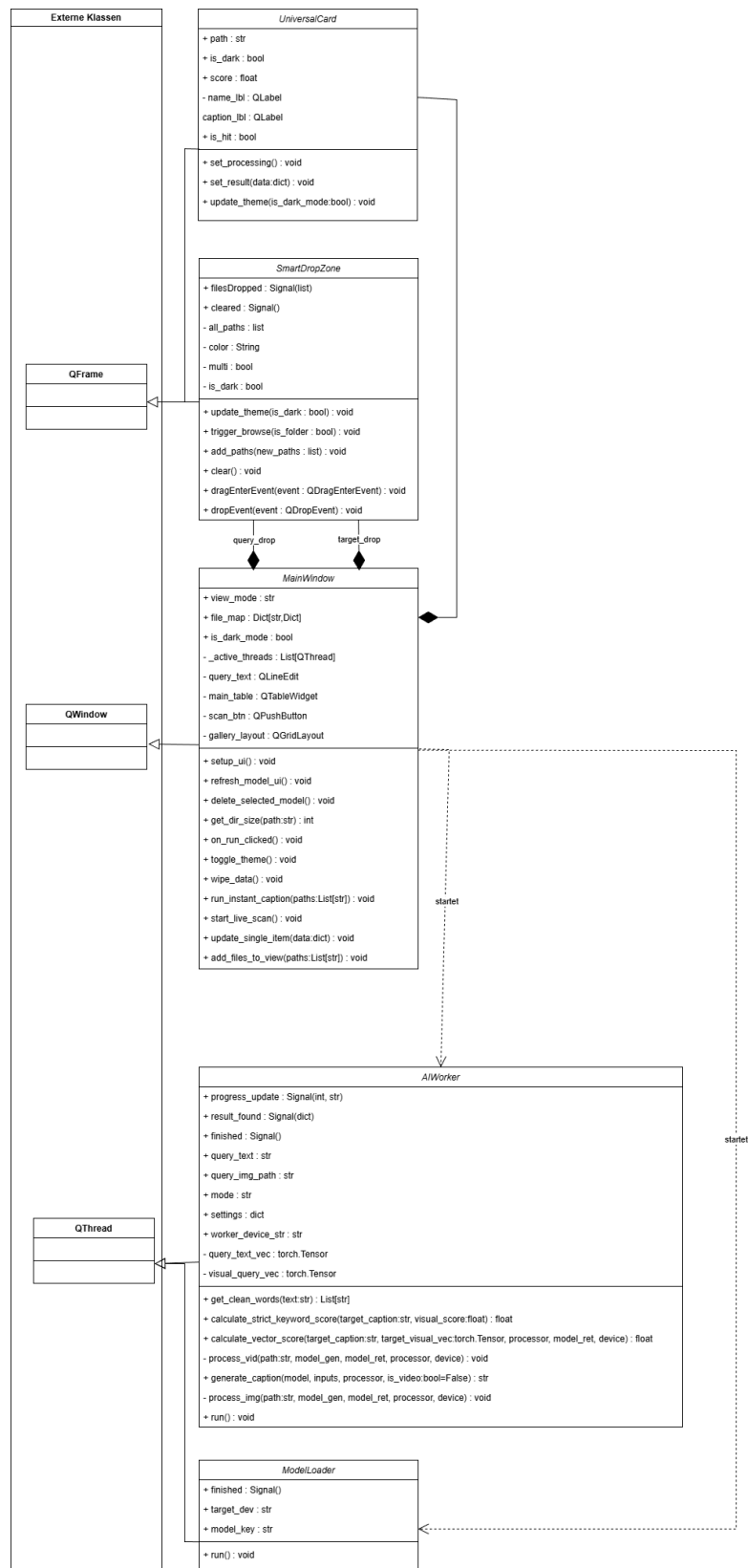


Abbildung 6: UML-Klassendiagramm

Das Klassendiagramm (Abbildung 6) beschreibt die statische Struktur der Anwendung und die Beziehungen der Kernklassen zueinander:

- **MainWindow:** ist für den Hauptteil der Benutzeroberfläche und deren Logik verantwortlich, wie zum Beispiel den Start eines Scans, das Löschen der Medien, den Wechsel zwischen Darkmode und Lightmode sowie das Anzeigen der Medien. Es erbt von der externen Klasse `QMainWindow` (PySide6), welche es erlaubt, ein vollwertiges Anwendungsfenster zu erstellen. Zur Verwaltung speichert es die einzelnen Medien und deren Metadaten im Dictionary `file_map`. Die Variable `view_mode` legt fest, ob die Medien aktuell als Liste oder als Galerie angezeigt werden und welches Theme die app hat über `toggle_theme()`. Außerdem verwaltet es Hintergrundprozesse in der Liste `_active_threads`.
- **SmartDropZone:** ist ein spezialisierter Rahmen, der von `QFrame` (PySide6) erbt, um die Drag-and-Drop-Logik zu ermöglichen. Sie verfügt über zwei Instanzen im MainWindow: `query_drop` ist das Eingabefeld für das Suchbild und `target_drop` das Eingabefeld für die Ziel-Dateien oder Ordner. Über das Signal `filesDropped` meldet sie neu hinzugefügte Pfade an das System weiter. Die interne Variable `is_dark` sorgt dafür, dass sich die Optik der Zone automatisch an das gewählte Theme des Hauptfensters anpasst.
- **UniversalCard:** ist die grafische Repräsentation eines einzelnen Ergebnisses und wird vom MainWindow erzeugt und verwaltet. Jede Karte speichert den Pfad zur Datei in `path`, den berechneten Ähnlichkeitswert in `score` sowie Informationen wie `is_hit`, um den Status der Suche anzuzeigen.
- **AIWorker:** Übernimmt die eigentliche Rechenarbeit und erbt von der Basisklasse `QThread` (PySide6), um parallel zum Hauptfenster zu laufen. Der Worker erhält seine Eingabedaten, wie beispielsweise `query_img_path` oder `query_text`, sowie die notwendigen `settings` direkt nach dem Start vom MainWindow. Er existiert als aktiver Prozess nur für die Dauer seiner Berechnung. Während des Laufs signalisiert er über `progress_update` den aktuellen Fortschritt an das MainWindow und berechnet das Ergebnis je nach Sucheinstellung über Funktionen wie `calculate_strict_keyword_score`, `calculate_vector_score` bzw. generiert den AI Prompt über `generate_caption`.
- **ModelLoader:** Identifiziert die benötigten KI-Modelle anhand des `model_key` und lädt diese auf das festgelegte Zielgerät `target_dev` (CPU oder GPU). Muss vor dem `AIWorker` ausgeführt werden, um die Betriebsbereitschaft der Modelle sicherzustellen.

2.1.4 Verhaltensübersicht

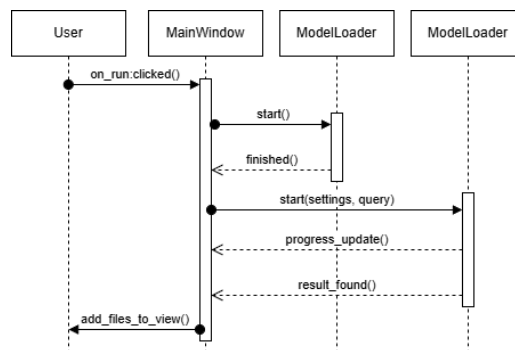


Abbildung 7: UML-Sequenzdiagramm

Das Sequenzdiagramm (Abbildung 7) beschreibt den zeitlichen Ablauf einer Suche:

Sobald der Nutzer den `scan_btn` drückt, ruft das `MainWindow` die Methode `on_run_clicked()` auf. Vor dem Start der eigentlichen Berechnung aktiviert das `MainWindow` den `ModelLoader` und übergibt ihm den `model_key` sowie das Zielgerät `target_dev` (CPU oder GPU). Dieser prüft über `get_engine_safe()`, ob das korrekte Modell bereits in `_GLOBAL_ENGINE` geladen ist, und lädt es andernfalls herunter bzw. initialisiert es neu. Erst wenn der `ModelLoader` über das Signal `finished` meldet, dass das Modell bereit ist, startet das `MainWindow` den `AIWorker` mit den vollständigen Suchparametern.

Während der `AIWorker` im Hintergrund die Rechenarbeit erledigt, sendet er über das Signal `progress_update` laufend den aktuellen Verarbeitungsstand an das Hauptfenster, das diesen in der Statusleiste visualisiert (NFA-02, R_2). Sobald die Analyse abgeschlossen ist, schickt der `AIWorker` die Endergebnisse über das Signal `result_found` an das `MainWindow` zurück, welches daraufhin `add_files_to_view()` aufruft und die Ergebnisse in der Galerie oder Tabellenansicht darstellt.

2.2 GitHub

- **GitHub Repository:** https://github.com/neli1212/Software_Entwicklung
- **exe der Reflektionsphase:** https://github.com/neli1212/Software_Entwicklung/releases/tag/v1.0.35

2.3 Benutzeranleitung

2.3.1 Semantische Suche für lokale Medien

Diese Anwendung soll die inhaltliche Durchsuchung lokaler Bild- und Videodateien per Text-Prompt oder Referenzbild ermöglichen. Durch lokale Modelle findet der gesamte Abgleich auf der eigenen GPU/CPU statt, wodurch die Daten privat bleiben und keine Cloud-Anbindung nötig ist.

Die kompilierte `.exe` ist auf GitHub unter **Releases** im Abschnitt **Erarbeitungs- und Reflektionsphase** zu finden. Es gibt zwei Varianten: eine ohne CUDA (CPU) und eine mit CUDA. Funktional sind beide identisch, jedoch kann die CUDA-Version die GPU nutzen und ist dadurch deutlich schneller. Da CUDA zu groß für GitHub ist, kann es nicht direkt in die kompilierte Datei integriert werden. Deshalb muss bei der CUDA-Version zuerst die mitgelieferte Setup-Datei für CUDA ausgeführt werden. Bei der Version ohne CUDA reicht es aus, das Archiv zu entpacken und die `.exe` zu starten.

Beim ersten Start muss das Programm mindestens ein Modell herunterladen und speichern. Dafür benötigt es Schreibrechte auf seinen eigenen Ordner. Falls diese nicht vorhanden sind, müssen sie gegeben werden, zum Beispiel indem die `.exe` als Administrator ausgeführt wird oder indem dem Ordner Schreibrechte gegeben werden. Wenn sich die Anwendung z. B. in „C:\Program Files\AISearchEngine“ befindet, kann dies über PowerShell (als Administrator) mit folgendem Befehl erfolgen:

```
icacls "C:\Program Files\AISearchEngine" /grant *S-1-5-32-545:(OI)(CI)M
```

2.3.2 Eingabe

Unter **AI Prompt & Image** kann je nach Suchmodus der zu suchende Begriff eingegeben werden oder ein Bild ausgewählt werden. Aus diesem Bild wird automatisch ein AI-Prompt generiert, den man anschließend selbst noch bearbeiten kann.

Unter **Target Data** sind die zu durchsuchenden Dateien bzw. Ordner einzufügen.

2.3.3 Suche

Unter **AI Settings** kann man unter **Comparison Logic** den Suchmodus einstellen. Entweder wird nach Keywords in den generierten AI-Prompts gesucht oder es werden die beiden Vektoren der Eingabe (Text oder Bild) und der zu durchsuchenden Dateien miteinander verglichen.

Aktuell kann man in **KI-Modell Version** zwischen zwei Modellen auswählen, die sich hauptsächlich in ihrer Größe unterscheiden.

Zusätzlich kann man die Generierung der Prompts über einige Parameter beeinflussen:

- **Beam Size** bestimmt, wie viele mögliche Textvarianten das Modell gleichzeitig berechnet und miteinander vergleicht.
- **Min Length** legt fest, wie lang der generierte Prompt mindestens sein muss, damit das Modell nicht zu kurze Beschreibungen erzeugt; zu lange lässt das Modell allerdings halluzinieren.
- **Length Penalty** beeinflusst, ob das Modell eher kürzere oder längere Beschreibungen generiert. Je nach Wert werden längere oder kürzere Texte bevorzugt.
- **No Repeat** bestraft, wenn Wortfolgen mehrfach hintereinander generiert werden, und reduziert so Wiederholungen im Prompt.

Bei der ersten Suche für ein Modell wird dieses automatisch heruntergeladen, was etwas dauern kann. Die Modelle sollten anschließend im selben Verzeichnis wie die `.exe` unter dem Ordner **AI_models** auffindbar sein.

3 Finalisierungsphase

3.1 Teststrategie

Um die Qualität des Systems zu sichern, werden Testfälle eingefügt, die automatisch über einen GitHub-Workflow ausgeführt werden; dabei gibt es drei Arten von Testfällen:

- **Unit-Tests:** Unit-Tests validieren einzelne Funktionen isoliert. Die Testfälle UT-01 bis UT-05 überprüfen die Medien-Indizierung durch die Funktion `collect_all_media`, was

die Grundlage für US-01 bildet. Die Bereinigung und Aufteilung von Suchbegriffen (UT-06 bis UT-09) sowie das exakte Scoring-Verfahren zur Ermittlung der Trefferrelevanz (UT-10 bis UT-13) stellen die korrekte Funktionsweise der Textsuche nach US-03 sicher. UT-14 und UT-15 sollen das Ressourcenmanagement gemäß US-07 absichern, indem der Download-Status der KI-Modelle geprüft wird, während UT-16 und UT-17 die notwendigen lokalen Dateisystemberechtigungen kontrollieren.

- **Integrationstests:** Integrationstests prüfen das Zusammenspiel mehrerer Komponenten. Der Testfall IT-01 validiert das Laden und Anzeigen von Medien in der Tabelle und Galerie (US-01), während IT-02 das vollständige Leeren dieser Datenstrukturen kontrolliert. Die fehlerfreie Aktualisierung der UI-Zellen mit dem berechneten Score und der generierten Bildbeschreibung wird durch IT-04 abgedeckt, was die Anforderungen an die Textsuche (US-03) und die Nachvollziehbarkeit der Ergebnisse (US-05) erfüllt. IT-05 stellt sicher, dass Inferenzprozesse über einen separaten Background-Thread entkoppelt werden, um ein Einfrieren der GUI zu verhindern (NFA-02). Die korrekte Übergabe des jeweils gewählten Suchmodus (Keyword- oder Vektormodus) an diesen Thread wird durch IT-06 und IT-07 verifiziert (US-02/US-03).
- **Systemtests:** Systemtests prüfen die Anwendung als Gesamtsystem über automatisierte Benutzeroberflächen-Interaktionen aus Sicht des Endanwenders. ST-01 validiert den korrekten Anwendungsstart und die allgemeine Bedienbarkeit. Der vollständige Ablauf beim Hinzufügen einer Datei durch den Nutzer wird über ST-02 simuliert und sichert US-01 ab. Das Abfangen leerer Suchanfragen mittels einer Fehlermeldung wird durch ST-03 getestet, um die Systemstabilität und die modulare Fehlertoleranz nach NFA-04 zu unterstützen. ST-04 prüft den manuellen Wechsel des KI-Modells über die Benutzeroberfläche zur Skalierung von Speicherplatz und Rechenleistung (US-07). Schließlich verifiziert ST-06 den gesamten End-to-End-Such-Workflow von der Dateiauswahl und der Prompt-Eingabe bis hin zur korrekten Ergebnisanzeige, wodurch das Zusammenwirken von US-01, US-03 und US-05 final nachgewiesen wird.

3.2 Unit-Tests

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
UT-01	collect_all_ media	Gültige Bild- und Videodatei- en werden erkannt.	—	[str(img), str(vid)]	len(result) == 2
UT-02	collect_all_ media	Ungültige Dateiformate wer- den ignoriert.	—	[str(txt), str(pdf)]	len(result) == 0
UT-03	collect_all_ media	Verzeichnisse werden rekur- siv durchsucht (US-01).	Unterordner mit pic1.png; Stammordner mit pic2.jpeg und ignore.txt.	[str(tmp_path)]	len(result) == 2
UT-04	collect_all_ media	Duplikate werden entfernt.	Dieselbe Datei wird direkt und über den Ordner übergeben.	[str(img), str(img), str(tmp_path)]	len(result) == 1
UT-05	collect_all_ media	Leere Eingabe ergibt leere Rückgabe.	—	[]	result == []
UT-06	get_clean_ words	Text wird korrekt in Kleinbuch- staben aufgeteilt.	—	"Hello World"	["hello", "world"]
UT-07	get_clean_ words	Englische Stoppwörter wer- den entfernt.	—	"The quick brown fox is in the box"	["quick", "brown", "fox", "box"]

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
UT-08	get_clean_ words	Satzzeichen werden ignoriert.	—	"Cat, dog! Mouse?"	["cat", "dog", "mouse"]
UT-09	get_clean_ words	Zahlen werden als gültige Wörter behandelt.	—	„I have 2 cats"	[„i", "have", "2", "cats"]
UT-10	calculate_ strict_ keyword_score	Perfekter Treffer ergibt Score 1,0.	query_words = ["gray", "cat"]	„A gray cat is sitting", visual_score = 0.0	score == 1.0
UT-11	calculate_ strict_ keyword_score	Teilweiser Treffer über 50 % ergibt Score 0,60.	query_words = ["gray", "cat", "sleeping"]	„A gray cat is awake", visual_score = 0.0	round(score, 2) == 0.60
UT-12	calculate_ strict_ keyword_score	Kein Treffer ergibt Score 0,0.	query_words = ["dog"]	„A gray cat", visual_ score = 0.5	score == 0.0
UT-13	calculate_ strict_ keyword_score	Leere Query gibt visual_ score zurück.	query_words = []	„A gray cat", visual_ score = 0.8	score == 0.8

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
UT-14	check_model_downloaded	Gibt True zurück wenn Modelldordner existiert und Dateien enthält.	Modelldordner: ["config.json", "pytorch_model.bin"]	model_key = "Base"	True
UT-15	check_model_downloaded	Gibt False zurück wenn Modelldordner leer ist.	Modelldordner: []	model_key = "Base "	False
UT-16	check_permissions	Gibt True zurück wenn Schreibzugriff besteht.	Schreibzugriff erteilt	—	True
UT-17	check_permissions	Gibt False zurück wenn kein Schreibzugriff besteht.	Schreibzugriff nicht erteilt.	—	False

3.3 Integrationstests

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
IT-01	add_files_to_ view	Testbild wird korrekt in Tabelle und Galerie geladen (US-01).	Testbild existiert	[test_image_path]	rowCount() == 1, len(file_map) == 1, gallery_layout.count() == 1, Dateiname in Tabellenzelle.
IT-02	wipe_data	Tabelle und file_map werden nach Bereinigung vollständig geleert.	Testbild geladen; rowCount() == 1.	Aufruf von wipe_data()	rowCount() == 0, len(file_map) == 0.
IT-04	update_ single_item	KI-Ergebnis aktualisiert Zeilen korrekt (BR-04).	Testbild geladen; file_map enthält Eintrag.	{'path': test_image_path, 'score': 0.85, 'caption': „Eine Katze“}	Tabelle zeigt "85.0%" und Eine Katze"; card.score == 0.85; card.is_hit is True.
IT-05	start_live_ scan	Scan-Start erzeugt einen AIWorker und fügt ihn zu _active_threads hinzu.	Testbild geladen; models_loaded = True; AIWorker gemockt.	query_text = "Katze", Aufruf von start_live_scan()	AIWorker einmal instanziiert; len(_active_threads) == 1.

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
IT-06	start_live_-scan	Keyword-Modus wird korrekt an AIWorker übergeben (BR-03).	Testbild geladen; models_-loaded = True; AIWorker gemockt; Sucheinstellung ist Keyword Match	query_text = "Katze", Aufruf von start_live_-scan()	settings["mode"] == "keyword"
IT-07	start_live_-scan	Vector-Modus wird korrekt an AIWorker übergeben (BR-03).	Testbild geladen; models_-loaded = True; AIWorker gemockt; Sucheinstellung ist Vector Space	query_text = "Katze", Aufruf von start_live_-scan()	settings["mode"] == "vector"

3.4 Systemtests

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
ST-01	MainWindow	Anwendung startet und Hauptfenster ist bedienbar.	—	Aufruf window.show()	von isVisible() ist True; scan_btn.isEnabled() ist True.
ST-02	add_files_to_view	Benutzer fügt Datei hinzu; sie erscheint in der Oberfläche (US-01).	Testbild	[test_image_path]	len(file_map) == 1; rowCount() == 1.
ST-03	scan_btn	Suche ohne Query zeigt Fehlermeldung statt Absturz (NFA-04).	Testbild gela- den; models_ loaded = True; QMessageBox.warning gemockt.	Klick auf scan_btn ohne Texteingabe.	QMessageBox.warning wurde aufgerufen.

Testfall-ID	Funktion	Beschreibung	Vorbedingung	Eingabe	Erwartetes Ergebnis
ST-04	combo_ai_model	Benutzer kann zwischen verfügbaren Modellen wechseln (US-07).	Mindestens zwei Modelle in combo_ai_model vorhanden.	combo.setCurrentIndex(1)currentText()	unter-scheidet sich vom vorherigen Wert.
ST-06	Gesamter Such-Workflow	Datei laden, Query eingeben, Suche starten, Ergebnis erscheint.	Testbild geladen; models_loaded = True; AIWorker gemockt.	query_text = „Eine Katze“, Klick auf scan_btn; update_single_item mit score = 0.95	worker.start aufgerufen; Tabelle zeigt "95.0%".

Literatur

- Atlassian (2026). Risikomatrix. <https://www.atlassian.com/de/work-management/project-management/risk-matrix>.
- Iman, M., Arabnia, H. R., and Rasheed, K. (2023). A review of deep transfer learning and recent advancements. *Technologies*, 11(2):40. <https://doi.org/10.3390/technologies11020040>.
- Li, J., Li, D., Xiong, C., and Hoi, S. (2022). BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 12888–12900. PMLR. <https://arxiv.org/abs/2201.12086>.
- Liang, W., Zhang, Y., Kwon, Y., Yeung, S., and Zou, J. (2022). Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=S7Evzt9uit3>.
- Marmsoler, D., Malkis, A., and Eckhardt, J. (2015). A model of layered architectures. *Electronic Proceedings in Theoretical Computer Science*, 178:47–61. <https://arxiv.org/pdf/1503.04916.pdf>.
- OpenCV Team (2026). Open source computer vision library. <https://opencv.org/>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gilmelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8024–8035. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- Pillow Contributors (2026). Pillow (pil fork) documentation. <https://pillow.readthedocs.io/en/stable/>.
- PyTorch Contributors (2024). PyTorch: An open source machine learning framework. <https://pytorch.org>.

- Qamar, T. and Bawany, N. Z. (2023). Understanding the black-box: Towards interpretable and reliable deep learning models. *PeerJ Computer Science*, 9:e1629. <https://doi.org/10.7717/peerj-cs.1629>.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR. <https://arxiv.org/abs/2103.00020>.
- The Qt Company (2024). Qt for Python (PySide6) – official python bindings. <https://www.qt.io/qt-for-python>.